SELF-ADAPTIVE MULTI-AGENT RETRIEVAL-AUGMENTED GENERATION (RAG) FRAMEWORK FOR DOMAIN-SPECIFIC DECISION SUPPORT

A PROJECT REPORT SUBMITTED TO

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

MASTER OF COMPUTER APPLICATIONS

BY

VISAGAN M (RA2532241010413)
ASHWIN S (RA2532241010365)
ABDUL-BASITH H (RA2532241010420)
MUKILAN A (RA2532241010387)

UNDER THE GUIDANCE OF

Dr. ANWAR R SHAHEEN
MBA., MCA., BEd., M.Phil., PhD



DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Kattankulathur – 603 203, Chengalpattu District, Tamil Nadu

OCTOBER - 2026

OCTOBER - 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"SELF-ADAPTIVE MULTI-AGENT RETRIEVAL-AUGMENTED GENERATION (RAG) FRAMEWORK FOR DOMAIN-SPECIFIC DECISION SUPPORT"** is a bonafide work carried out by **VISAGAN M (RA2532241010413)**, **ASHWIN S (RA2532241010365)**, **ABDUL-BASITH H (RA2532241010420)**, and **MUKILAN A (RA2532241010387)** under my supervision for the award of the Degree of **Master of Computer Applications**. To the best of my knowledge, the work reported herein is the original work carried out by these candidates.

Dr. ANWAR R SHAHEEN
MBA., MCA., BEd., M.Phil., PhD
Department of Computer Applications
(GUIDE)

Dr. P. J. ARUL LEENA ROSE
Professor & Head,
Department of Computer Applications
SRM Institute of Science and Technology

Submitted for the Project Viva-Voce Examination held on: _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, we take this chance to thank the people who helped us to complete this project successfully.

We take this as a right opportunity to say THANKS to our parents who are there to stand with us always with the words “YOU CAN”.

We are thankful to **Dr. T. R. Paarivendhar**, Chancellor, SRM Institute of Science & Technology, who gave us the platform to establish ourselves to reach greater heights.

We earnestly thank **Dr. A. Duraisamy**, Dean, Faculty of Science and Humanities, SRM Institute of Science & Technology, who always encourages us to do novel things.

A great note of gratitude to **Dr. S. Albert Antony Raj**, Deputy Dean, Faculty of Science and Humanities, for his valuable guidance and constant support to do this project.

We express our sincere thanks to **Dr. P. J. Arul Leena Rose**, Professor & Head, Department of Computer Applications, for her continuous support and academic encouragement.

It is our delight to thank our project guide **Dr. ANWAR R SHAHEEN**, MBA., MCA., BEd., M.Phil., PhD , Department of Computer Applications for her help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

We convey our gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during reviews.

Our gratitude to friends and peers who directly and indirectly helped in carrying out this project work successfully.

Visagan M (RA2532241010413)

Ashwin S (RA2532241010365)

Abdul-Basith H (RA2532241010420)

Mukilan A (RA2532241010387)

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
	Abstract	**v**
	List of Tables	**x**
	List of Figures	**xi**
1	**Introduction**	**1**
1.1	Background of the Study	1
1.2	Problem Statement	1
1.3	Objectives of the Project	2
1.4	Scope of the Project	2
1.5	Significance of the Project	3
2.	**Literature Survey / Existing Studies**	**4**
2.1	Review of Existing Systems and Research Works	4
2.2	Comparative Analysis	5
2.3	Research Gap and Limitations of Existing Systems	5
2.4	Proposed System and Proposed Approach	6
3.	**Software Requirement Analysis**	**7**
3.1	Hardware Specification	7
3.2	Software Specification	8
3.3	Software Tools and Their Features	9
4.	**System Analysis**	**11**
4.1	Requirement Specification	11
4.2	Characteristics of Existing System	12

TABLE OF CONTENTS (Continued)

Sl. No.	Contents	Page No.
4.3	Proposed System	12
4.4	Feasibility Study	13
4.5	Software Requirement Specification (SRS)	14
5.	**System Design**	**16**
5.1	System Architecture and Overall Workflow	16
5.2	Context Diagram (DFD Level 0)	17
5.3	Use Case Diagram	18
5.4	Activity Diagram / State Transition Flow	19
5.5	Data Flow Diagram (DFD Level 1)	20
5.6	Module Design & State Attributes Schema	22
5.7	User Interface Design	23
6.	**System Implementation**	**25**
6.1	Module Description	25
6.2	Dataset Description and Data Collection	27
6.3	Data Preprocessing and Chunking Pipeline	28
6.4	Feature Engineering and Dense Embeddings	29
6.5	Model Architecture and Algorithm Design	30
6.6	Implementation of Routing & Retrieval System	31
6.7	Implementation of Web Search & Synthesis	32
6.8	Integration and Execution of the Application	33
7.	**Testing and Evaluation**	**34**
7.1	Introduction to Testing and Test Case Design	34
7.2	Unit Testing of Agentic Modules	35
7.3	Integration Testing of Multi-Agent Cycles	37
7.4	System Testing	38
7.5	Model and Performance Evaluation (RAG Triad)	39
7.6	Comparative Analysis (Basic vs Static vs Adaptive)	41

TABLE OF CONTENTS (Continued)

Sl. No.	Contents	Page No.
8.	**Results and Conclusion**	**43**
8.1	Results	43
8.2	Discussion and Analysis	45
8.3	Limitations	46
8.4	Lessons Learnt	47
8.5	Conclusion	48
8.6	Future Enhancements	49
9.	**Appendices**	**51**
9.1	Introduction to Appendices	51
9.2	Application Screenshots & System Outputs	51
9.3	Selected Source Code	54
9.4	Installation and Setup Instructions	55
9.5	User Documentation and Manual	56
9.6	Glossary of Technical Terms	58
10.	**Bibliography / References**	**61**
10.1	References	61
10.2	Relevance of the References	61
10.3	Conclusion of the Study	62

ABSTRACT

The deployment of Large Language Models (LLMs) in real-world advisory systems is constrained by severe operational limitations, notably factual hallucinations, static knowledge cutoffs, context saturation, and an inability to synthesize interdisciplinary technical problems reliably. While conventional Retrieval-Augmented Generation (RAG) mitigates cutoffs by fetching relevant passages from a vector database, standard monolithic architectures enforce an unvarying, single-pass pipeline regardless of inquiry difficulty. Consequently, simple questions incur unnecessary latency and token costs, while complex multi-criteria inquiries suffer from cross-domain semantic confusion, zero claim verification, and unverifiable answers.

The Self-Adaptive Multi-Agent Retrieval-Augmented Generation (RAG) Framework is an intelligent decision-support web platform engineered to overcome these challenges. The system introduces an autonomous, multi-agent network orchestrated via a stateful LangGraph engine that evaluates user queries across five critical domains: Agriculture, Finance, Energy, Healthcare, and Education. Instead of executing an unvarying chain, the framework features an adaptive complexity classifier that classifies queries into Low, Medium, or High complexity and dynamically provisions processing pipelines.

The framework partitions enterprise knowledge into distinct ChromaDB vector collections powered by local dense embeddings (all-MiniLM-L6-v2). Domain-specific autonomous agents (Agriculture Agent, Finance Agent, Energy Agent, Healthcare Agent, and Education Agent) conduct specialized analyses grounded exclusively on retrieved vector chunks. For medium and high-complexity queries, an autonomous Verification Agent performs rigorous fact-checking against raw source chunks, identifying unsupported claims, resolving conflicting statements, and outputting an algorithmic alignment score. A Confidence Evaluation Engine computes a composite evidence reliability score; if confidence drops below 60%, the coordinator triggers an iterative re-retrieval loop.

The application incorporates a resilient, multi-provider LLM interface powered by OpenRouter, dynamically routing requests to active free models (openrouter/free, google/gemma-4-31b-it:free, qwen/qwen3.8-27b:free) or commercial flagship models (openai/gpt-4o). To guarantee uninterrupted demonstration and zero service failures during network outages or key exhaustion, the architecture includes an internal Extractive Knowledge Synthesizer as an automated offline fallback.

The user interface, developed in Streamlit, replaces raw technical logs with a structured Google AI Overview layout. The interface organizes answers sequentially: Direct Answer, Understand It Simply, Visual Operational Flow, Key Points, Decision Guide, Evidence-supported Advantages and Limitations, and Source Receipts with cosine similarity metrics. The framework was evaluated across 40 rigorous test scenarios. Results indicate that adaptive routing reduces inference overhead on simple queries by up to 68% while elevating response faithfulness and citation precision to over 92% on complex inquiries, delivering an end-to-end decision-support assistant for non-technical users.

Future enhancements may include integration with enterprise relational databases, distributed vector sharding across terabyte corpora, continuous active learning loops from user feedback, and role-based access control (RBAC). The system is intended as an autonomous decision-support prototype and demonstrates verifiable evidence grounding across diverse multidisciplinary operational environments.

Keywords: Artificial Intelligence, Multi-Agent Systems, Retrieval-Augmented Generation (RAG), LangGraph, Vector Databases, ChromaDB, Decision Support Systems, Automated Fact Verification, OpenRouter, Streamlit.

PLAGIARISM CERTIFICATE

SmallSEOTools

Plagiarism Detection Report by SmallSEOTOOLS



Plagiarism	0%	Partial Match	0%
Exact Match	0%	Unique	100%

Scan details

Total Words	Total Characters	Plagiarized Sentences	Unique Sentences
455	3852	0	19 (100%)

#1 100% Unique

The deployment of Large Language Models (LLMs) in real-world advisory systems is constrained by severe operational limitations, notably factual hallucinations, static knowledge cutoffs, context saturation, and an inability to synthesize interdisciplinary technical problems reliably. While conventional Retrieval-Augmented Generation (RAG) mitigates cutoffs by fetching relevant passages from a vector database, standard monolithic architectures enforce an unvarying, single-pass pipeline regardless of inquiry difficulty. Consequently, simple questions incur unnecessary latency and token costs, while complex multi-criteria inquiries suffer from cross-domain semantic confusion, zero claim verification, and unverifiable answers.

The Self-Adaptive Multi-Agent Retrieval-Augmented Generation (RAG) Framework is an intelligent decision-support web platform engineered to overcome these challenges. The system introduces an autonomous, multi-agent network orchestrated via a stateful LangGraph engine that evaluates user queries across five critical domains: Agriculture, Finance, Energy, Healthcare, and Education. Instead of executing an unvarying chain, the framework features an adaptive complexity classifier that classifies queries into Low, Medium, or High complexity and dynamically provisions processing pipelines.

The framework partitions enterprise knowledge into distinct ChromaDB vector collections powered by local dense embeddings (all-MiniLM-L6-v2). Domain-specific autonomous agents (Agriculture Agent, Finance Agent, Energy Agent, Healthcare Agent, and Education Agent) conduct specialized analyses grounded exclusively on retrieved vector chunks. For medium and high-complexity queries, an autonomous Verification Agent performs rigorous fact-checking against raw source chunks, identifying unsupported claims, resolving conflicting statements, and outputting an algorithmic alignment score. A Confidence Evaluation Engine computes a composite evidence reliability score; if confidence drops below 60%, the coordinator triggers an iterative re-retrieval loop.

The application incorporates a resilient, multi-provider LLM interface powered by OpenRouter, dynamically routing requests

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Hardware Requirements	7
Table 3.2	Software Specification and Tooling Stack	8
Table 5.1	Vector Store and LangGraph State Data Schema	23
Table 7.1	Functional Test Cases and Expected Results	34
Table 7.2	Unit Testing Record Across Agent Modules	37
Table 7.3	Integration Test Cases for State Machine Transitions	37
Table 7.4	System Testing Checklist	38
Table 7.5	Performance Evaluation Record	40
Table 7.6	Qualitative Evaluation of the Framework	41
Table 8.1	Summary of System Functional Results	43

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 5.1	System Architecture of the Proposed Framework	17
Figure 5.2	Context Diagram of the Proposed System (DFD Level 0)	18
Figure 5.3	Use Case Diagram of the Proposed System	19
Figure 5.4	Activity Diagram of the Proposed System	20
Figure 5.5	Data Flow Diagram of the Proposed System (DFD Level 1)	21
Figure 5.6	Streamlit User Interface / Application Output Wireframe	24
Figure 6.1	Multi-Agent Routing and Retrieval Sequence Flowchart	31
Figure 6.2	Multi-Source Context Fusion & Fallback Synthesis Flowchart	32
Figure 8.1	LangGraph Multi-Agent Execution State Graph	44
Figure 8.2	RAG Triad Evaluation Metrics Radar and Distribution	45
Figure 8.3	Execution Latency Comparison Across Modes	45
Figure A.1	Application Execution -- Terminal to Run the Program	51
Figure A.2	Streamlit Web Interface -- Giving User Query	52
Figure A.3	Streamlit Web Interface -- Output that the Query Gives	53
Figure A.4	End-to-End System Integration Flowchart	54